UNIVERSIDAD GALILEO

TECNICO EN DESARROLLO DE SOFTWARE

SISTEMA DE GESTION DE INVENTARIO CON INTEGRACIÓN SQL Y NOSQL

Curso: Base de Datos

Catedrático: Mynor Ruiz

Integrantes:

25001309 Rafael Humberto Morales Lara

25003028 Kevin Estuardo Menéndez Lara

25004369 Erick Javier Cabrera Morales

25004942Vyanka Ivette Morales Gómez

Ciudad de Guatemala, marzo 2026

Contenido

Introducción 4
Objetivo del proyecto 4
Alcance 4
Descripción del sistema 4
Requerimientos 5
Requerimientos Funcionales 5
Requerimientos No Funcionales 6
Diseño del sistema 7
Diagrama Entidad-Relación 7
Explicación Relaciones 7
Tabla de relaciones 8
Diseño Lógico 10
Composición de las tablas 15
Implementación en MySQL Workbench 16
Creación base de datos: 16
Creación tabla categoría: 16
Creación tabla rol: 16
Creación tabla empleado: 16
Creación tabla empleado_auth: 17
Creación tabla producto: 18
Creación tabla almacén: 18
Creación tabla inventario: 18
Creación tabla venta: 19
Creación tabla detalle_venta: 19
Creación tabla transferencia: 20
Implementación de Triggers en SQL 21
Creación de índices 25
Creación de Vistas y Procedimientos 25
Vistas 27
Procedimientos Almacenados 34

Particiones 37
Horizontal 37
Vertical 37
MongoDB 38
Seleccionamos la base de datos 38
Creamos las colecciones 38
Insertamos documentos de prueba 39
Realizamos consultas básicas 39
Cambios Fase 2 41
Replicación MySQL 43
Ajustando my.ini del máster y creando el usuario de replicación. 43
Comandos ejecutados en el Máster 43
Configuración del Slave 44
Comandos ejecutados en el Slave 44
Configuración del Slave para conectarse al Máster 46
API 47
Justificación técnica 49
Limitaciones Técnicas 49
Conclusión 49
Presentación 49

Introducción

Este proyecto presenta el diseño e implementación de un sistema de gestión de inventario para una empresa, incluyendo el control de productos y ventas. El objetivo principal es organizar y generar reportes que faciliten la toma de decisiones.

Objetivo del proyecto

Desarrollar un sistema que permita:

- Analizar los requerimientos del sistema
- Diseñar el modelo conceptual y lógico de las bases de datos.
- Transformarlo a modelo lógico
- Implementar el sistema utilizando tecnologías SQL y NoSQL
- Integrar mecanismos de consulta, modificación y generación de reportes.

Alcance

El sistema permite gestionar inventarios y ventas para un mejor manejo sobre el producto. No incluye funcionalidades de facturación, clientes ni implementaciones con sistemas externos. Este es un sistema para generar reportes y consultas sobre un producto y por lo tanto se enfoca en:

- Registro y consulta de productos.
- Control de stock por almacén.
- Generación de reportes de inventarios y ventas.
- Almacenamiento histórico en MongoDB.

Descripción del sistema

Las personas que utilizarán este sistema son empleados de la empresa, que desean consultar acerca del stock de uno o mas productos, divididos en administradores y operadores, los administradores podrán modificar, y hacer cambios sobre la información del producto, los almacenes, y consultar los empleados de la empresa, incluyendo eliminar usuarios de empleados que ya no se encuentran dentro de la empresa. Los operadores, solo podrán consultar la información de los productos, pero no podrán realizar modificaciones sin previa autorización de algún administrador. El sistema maneja información específica de los productos y las ventas que disminuyen el stock, para permitir un control sobre esta información.

Requerimientos

Requerimientos Funcionales

RF, 01: Registro de productos: El sistema debe permitir el registro la de nuevos productos en el inventario, incluyendo la siguiente información:

- Código del producto
- Nombre del producto
- Descripción
- Precio Unitario
- Stock disponible
- Ubicación dentro del almacén

RF-02: Actualización de productos: El sistema debe dar la posibilidad de modificar la información de los productos existentes, incluyendo cambio de nombre, descripción precio o ubicación.

RF-03: Eliminación de productos: El sistema debe posibilitar que se puedan eliminar productos del sistema siempre que no están vinculados a transacciones activas.

RF-04: Verificación de duplicidad: El sistema debe contener mecanismos para prevenir la duplicidad de productos, usando el código del producto como identificador único.

RF-05: Gestión de inventario: El sistema debe permitir consultar el inventario disponible de los productos, incluyendo el stock existen y su ubicación dentro de los diferentes almacenes.

RF-06: Reporte de inventario general: El sistema debe generar un reporte general de inventario que incluya:

- Código del producto
- Nombre del producto
- Stock actual
- Precio Unitario
- Valor total del inventario por producto (stock por precio)

RF-07: Reporte de productos por ubicación: El sistema debe permitir generar reportes de productos agrupados, ya sea por almacén o ubicación, para facilitar la organización del inventario.

RF-08: Reporte de ventas: El sistema debe generar reportes de transacciones de ventas, incluyendo la siguiente información como mínimo:

- Producto vendido
- Cantidad
- Fecha de transacción
- Valor total de la venta

RF-09: Búsquedas y filtros: El sistema debe permitir realizar consultas avanzadas, especialmente los siguientes filtros:

- Búsqueda de productos por rango de precios
- Identificación de productos con stock menor a un nivel determinado
- Búsqueda de productos por ubicación específica

RF-10: Gestión de usuarios: El sistema debe permitir la gestión de usuarios, incluyendo registro y administración de cuentas dentro del sistema.

RF-11: Gestión de roles: El sistema debe implementar roles de usuario para controlar el acceso al sistema, siendo estos:

- Administrador
- Operador

RF-12: Registro histórico en base de datos No relacional o NoSQL: El sistema debe de almacenar información histórica en una base de datos NoSQL, siendo incluida la siguiente información en cada registro:

- Historial de transacciones
- Cambios realizados en productos
- Comentarios registrados por los operadores

Requerimientos No Funcionales

RNF-01 Rendimiento: El sistema debe permitir consultas y generación de reportes de manera eficiente, inclusive con grandes volúmenes de datos.

RNF-02: Seguridad: El sistema debe implementar roles y permisos para restringir el acceso a determinadas operaciones según el tipo de usuario.

RNF-03: Escalabilidad: El sistema debe permitir la escalabilidad de la base de datos mediante técnicas de replicación y partición, garantizando disponibilidad y rendimiento.

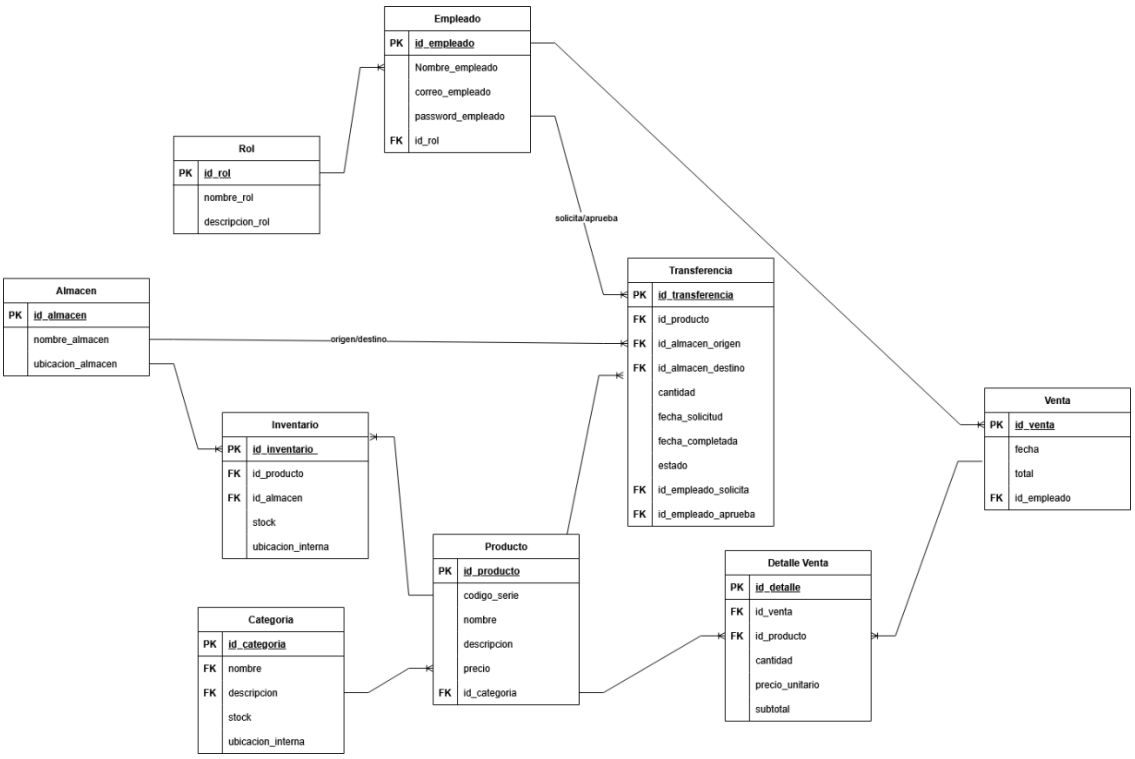
RNF-04: Integración de bases de datos: El sistema debe integrar bases de datos relaciones y no relacionales, dividiéndose en:

- MySQL para almacenamiento principal de datos.
- MongoDB para almacenamiento de registros históricos.

RNF-05: Disponibilidad: El sistema debe implementar replicación de datos para garantizar alta disponibilidad y tolerancia a fallos.

Diseño del sistema

Diagrama Entidad-Relación



Explicación Relaciones

Entidades:

- Rol
- Empleado
- Venta
- Detalle Venta
- Producto
- Inventario
- Almacen
- Categoria
- Empleado_auth
- Transferencia

Tabla de relaciones

Tabla 1	Relación	Tabla 2	Tipo	Descripción
rol	1 : N	empleado	Uno a muchos	Un rol puede tener muchos empleados
empleado	N : 1	rol	Muchos a uno	Cada empleado tiene un rol especifico
categoría	1 : N	producto	Uno a muchos	Una categoría agrupa muchos productos
producto	N : 1	categoría	Muchos a uno	Cada producto pertenece a una categoría
almacén	1 : N	inventario	Uno a muchos	Un almacén puede tener muchos productos en stock
producto	N : 1	inventario	Muchos a uno	Un producto puede estar en varios almacenes
inventario	N : 1	producto	Muchos a uno	Cada registro de inventario es de un producto
inventario	N : 1	almacén	Muchos a uno	Cada registro de inventario es de un almacén
empleado	1 : N	venta	Uno a muchos	Un empleado puede realizar muchas ventas
venta	N : 1	empleado	Muchos a uno	Cada venta es realizada por un empleado
almacén	1 : N	venta	Uno a muchos	Un almacén puede registrar muchas ventas
venta	N : 1	almacén	Muchos a uno	Cada venta se realiza en un almacén especifico
venta	1 : N	detalle_venta	Uno a muchos	Una venta puede tener varios productos
detalle_venta	N : 1	venta	Muchos a uno	Cada detalle pertenece a una venta
producto	1 : N	detalle_venta	Uno a muchos	Un producto puede aparecer en varias ventas
detalle_venta	N : 1	producto	Muchos a uno	Cada detalle corresponde a un producto
almacén	1 : N	detalle_venta	Uno a muchos	Un almacén puede tener muchas líneas de venta
detalle_venta	N : 1	almacén	Muchos a uno	Cada detalle de venta se registra en un almacén
producto	1 : N	transferencia	Uno a muchos	Un producto puede tener varias transferencias
transferencia	N : 1	producto	Muchos a uno	Cada transferencia es de un producto
almacén	1 : N	transferencia (origen)	Uno a muchos	Un almacén puede ser origen de varias transferencias
almacén	1 : N	transferencia (destino)	Uno a muchos	Un almacén puede ser destino de varias transferencias

empleado	1 : N	transferencia (solicita)	Uno a muchos	Un empleado puede solicitar varias transferencias
empleado	1 : N	transferencia (aprobada)	Uno a muchos	Un empleado puede aprobar varias transferencias
empleado	1 : 1	empleado_auth	Uno a uno	Cada empleado tiene datos de autenticación

Diseño Lógico

Tabla: Categoria

Campo	Tipo de Dato	Restricciones	Descripción
id_categoria	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador interno de la categoria
nombre	VARCHAR(50)	NOT NULL	Nombre de la categoria
descripcion	VARCHAR(100)		Descripcion breve de la categoria

Tabla: Rol

Campo	Tipo de Dato	Restricciones	Descripción
id_rol	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador interno del rol
nombre_rol	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol
descripcion_rol	VARCHAR(100)		Descripcion de las funciones del rol

Tabla: Almacen

Campo	Tipo de Dato	Restricciones	Descripción
id_almacen	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador interno del almacen
nombre_almacen	VARCHAR(50)	NOT NULL	Nombre del almacen
ubicacion_almacen	VARCHAR(80)	NOT NULL	Direccion fisica del almacen

Tabla: Producto			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
id_producto	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador interno del producto
codigo_serie	VARCHAR(50)	NOT NULL, UNIQUE	Codigo o serie del producto (SKU)
nombre	VARCHAR(50)	NOT NULL	Nombre del producto
descripcion	VARCHAR(80)		Breve descripcion del producto
precio	DECIMAL(10,2)	NOT NULL	Precio unitario del producto
id_categoria	INT	NOT NULL, FOREIGN KEY	FK a categoria(id_categoria)

Tabla: Inventario			
Campo	**Tipo de Dato**	**Restricciones**	**Descripcion**
id_inventario	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador del registro de inventario
id_producto	INT	NOT NULL, FOREIGN KEY	FK a producto(id_producto)
id_almacen	INT	NOT NULL, FOREIGN KEY	FK a almacen(id_almacen)
stock	INT	NOT NULL, DEFAULT 0	Cantidad disponible del producto
ubicacion_interna	VARCHAR(80)		Ubicacion exacta dentro del almacen
UNIQUE	(id_producto, id_almacen)		Un producto no puede duplicarse en el mismo almacen

Tabla: Empleado			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
id_empleado	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador interno del empleado
nombre_empleado	VARCHAR(60)	NOT NULL	Nombre completo del empleado
correo_empleado	VARCHAR(60)	NOT NULL, UNIQUE	Correo electronico del empleado
id_rol	INT	NOT NULL, FOREIGN KEY	FK a rol(id_rol)
activo	BOOLEAN	DEFAULT TRUE	Estado del empleado

Tabla: Empleado_Auth			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
id_empleado	INT	PRIMARY KEY, FOREIGN KEY	FK a empleado(id_empleado)
password_empleado	VARCHAR(255)	NOT NULL	Contrasena hasheada (SHA2-256)
ultimo_login	DATETIME	NULL	Fecha y hora del ultimo inicio de sesion
intentos_fallidos	INT	DEFAULT 0	Contador de intentos fallidos de login
bloqueado_hasta	DATETIME	NULL	Fecha hasta la cual el empleado esta bloqueado

Tabla: Venta			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
id_venta	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador interno de la venta
fecha	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora de la venta
total	DECIMAL(10,2)	NOT NULL	Total de la venta
id_empleado	INT	NOT NULL, FOREIGN KEY	FK a empleado(id_empleado)
id_almacen	INT	NOT NULL, FOREIGN KEY	FK a almacen(id_almacen)
estado	ENUM('pendiente','completada','cancelada')	DEFAULT 'pendiente'	Estado de la venta

Tabla: Detalle_Venta			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
id_detalle	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador unico del detalle
id_venta	INT	NOT NULL, FOREIGN KEY	FK a venta(id_venta)
id_producto	INT	NOT NULL, FOREIGN KEY	FK a producto(id_producto)
cantidad	INT	NOT NULL	Cantidad de productos vendidos
precio_unitario	DECIMAL(10,2)	NOT NULL	Precio al momento de la venta
id_almacen	INT	NOT NULL, FOREIGN KEY	FK a almacen(id_almacen)

Tabla: Transferencia

Campo	Tipo de Dato	Restricciones	Descripción
id_transferencia	INT (AUTO_INCREMENT)	PRIMARY KEY	Identificador de la transferencia
id_producto	INT	NOT NULL, FOREIGN KEY	FK a producto(id_producto)
id_almacen_origen	INT	NOT NULL, FOREIGN KEY	FK a almacen(id_almacen) - almacen de origen
id_almacen_destino	INT	NOT NULL, FOREIGN KEY	FK a almacen(id_almacen) - almacen de destino
cantidad	INT	NOT NULL, CHECK(cantidad > 0)	Cantidad a transferir
fecha_solicitud	DATETIME	DEFAULT CURRENT_TIMESTAMP	Fecha de solicitud de transferencia
fecha_completada	DATETIME	NULL	Fecha de finalizacion de transferencia
estado	ENUM('solicitada','aprobada','rechazada','completada','cancelada')	DEFAULT 'solicitada'	Estado de la transferencia
id_empleado_solicita	INT	NOT NULL,FOREIGN KEY	FK a empleado(id_empleado) quien solicita
id_empleado_aprueba	INT	NULL FOREIGN KEY	FK a empleado(id_empleado) quien aprueba
CHECK		(id_almacen_origen != id_almacen_destino)	No puede transferir al mismo almacen

Composición de las tablas

- **categoria**: id_categoria, nombre, descripción
- **rol**: id_rol, nombre_rol, descripcion_rol
- **almacen**: id_almacen, nombre_almacen, ubicacion_almacen
- **producto**: id_producto, codigo_serie, nombre, descripcion, precio, id_categoria
- **inventario**: id_inventario, id_producto, id_almacen, stock, ubicacion_interna
- **empleado**: id_empleado, nombre_empleado, correo_empleado, id_rol, activo
- **empleado_auth**: nueva tabla (separada del empleado), password_empleado, ultimo_login, intentos_fallidos, bloqueado_hasta
- **venta**: id_venta, fecha, total, id_empleado, id_almacen, estado
- **detalle_venta**: id_detalle, id_venta, id_producto, cantidad, precio_unitario, id_almacen
- **transferencia**: nueva tabla, id_producto, id_almacen_origen, id_almacen_destino, cantidad, fecha_solicitud, fecha_completada, estado, id_empleado_solicita, id_empleado_aprueba

Implementación en MySQL Workbench

Para implementar dentro de SQL Workbench, se utilizó el siguiente query para las tablas y base de datos:

Creación base de datos:

La base de datos debe ser creada para almacenar el inventario que va a contener el producto, la venta, el stock, para llevar el control que desea la empresa por sobre su producto.

```
DROP DATABASE IF EXISTS sistema_inventario;
CREATE DATABASE sistema_inventario;
USE sistema_inventario;
```

Creación tabla categoría:

Esta tabla fue creada para organizar productos por tipo, debido a esto se logra facilitar búsquedas y reportes por categoría y también permite realizar análisis de ventas por segmento.

```
DROP TABLE IF EXISTS categoria;
CREATE TABLE categoria (
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    descripcion VARCHAR(100)
);
```

Creación tabla rol:

No todos los empleados tienen el nivel de autorización para realizar cambios dentro de la base de datos, para poder diferenciarlos, se crea una tabla de roles, ahí se normaliza que tipo de acceso y responsabilidad tienen dentro de la empresa, un operador puede consultar las ventas, el inventario, etc, pero un administrador debería poder modificar información e inclusive tener control sobre quien puede consultar información más sensible.

```
DROP TABLE IF EXISTS rol;
CREATE TABLE rol (
    id_rol INT AUTO_INCREMENT PRIMARY KEY,
    nombre_rol VARCHAR(50) NOT NULL,
    descripcion_rol VARCHAR(100)
);
```

Creación tabla empleado:

Una empresa puede tener una gran variedad de empleados, y una tabla para llevar control de sus roles, su nombre y su correo, permiten una mejor comunicación entre los empleados, además de

evitar confusiones donde 2 empleados tengan nombres similares o casi idénticos, pero roles distintos.

```
DROP TABLE IF EXISTS empleado;
CREATE TABLE empleado (
    id_empleado INT AUTO_INCREMENT PRIMARY KEY,
    nombre_empleado VARCHAR(60) NOT NULL,
    correo_empleado VARCHAR(60) NOT NULL,
    id_rol INT NOT NULL,
    activo BOOLEAN DEFAULT TRUE,
    CONSTRAINT fk_empleado_rol FOREIGN KEY (id_rol) REFERENCES rol(id_rol),
    UNIQUE KEY correo_empleado (correo_empleado)
);
```

Creación tabla empleado_auth:

Durante la fase 2, se decidió hacer un particionamiento vertical de la tabla empleados, separando los datos de consulta de la autenticación de los datos operacionales del empleado, con el objetivo de aumentar la seguridad ya que ahora las contraseñas se almacenan de forma independiente, también permitiendo cambios en el sistema de autenticación sin que afecten a la tabla principal y mantener un mejor control de auditoría, ya que permite registrar intentos de login y bloqueos

```
DROP TABLE IF EXISTS empleado_auth;
CREATE TABLE empleado_auth (
    id_empleado INT PRIMARY KEY,
    password_empleado VARCHAR(255) NOT NULL,
    ultimo_login DATETIME NULL,
    intentos_fallidos INT DEFAULT 0,
    bloqueado_hasta DATETIME NULL,
    FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado)
);
```

Creación tabla producto:

La tabla producto, se encarga de proveer la información acerca de un producto específico, mostrando la disponibilidad.

```
DROP TABLE IF EXISTS producto;
CREATE TABLE producto (
    id_producto INT AUTO_INCREMENT PRIMARY KEY,
    codigo_serie VARCHAR(50) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    descripcion VARCHAR(80),
    precio DECIMAL(10,2) NOT NULL,
    id_categoria INT NOT NULL,
    UNIQUE KEY codigo_serie (codigo_serie),
    FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)
);
```

Creación tabla almacén:

Representa varios almacenes donde se almacenan los productos.

```
DROP TABLE IF EXISTS almacen;
CREATE TABLE almacen (
    id_almacen INT AUTO_INCREMENT PRIMARY KEY,
    nombre_almacen VARCHAR(50) NOT NULL,
    ubicacion_almacen VARCHAR(80) NOT NULL
);
```

Creación tabla inventario:

Para registrar la cantidad de productos disponibles en cada almacen.

```
DROP TABLE IF EXISTS inventario;
CREATE TABLE inventario (
    id_inventario INT AUTO_INCREMENT,
    id_producto INT NOT NULL,
    id_almacen INT NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    ubicacion_interna VARCHAR(80),
    PRIMARY KEY (id_inventario, id_almacen),
    UNIQUE KEY idx_producto_almacen (id_producto, id_almacen)
);
```

Creación tabla venta:

La tabla ventas, muestra la transacción que realizan los empleados al vender los productos, identificando la fecha, el total de la venta y el empleado que hizo dicha venta, esto para llevar un control de comisiones por venta.

```
DROP TABLE IF EXISTS venta;
CREATE TABLE venta (
    id_venta INT AUTO_INCREMENT PRIMARY KEY,
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    total DECIMAL(10,2) NOT NULL,
    id_empleado INT NOT NULL,
    id_almacen INT NOT NULL,
    estado ENUM('pendiente', 'completada', 'cancelada') DEFAULT 'pendiente',
    FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado),
    FOREIGN KEY (id_almacen) REFERENCES almacen(id_almacen)
);
```

Creación tabla detalle_venta:

La tabla de detalle_venta, almacena más detalladamente a la información de cada venta, permitiendo registrar varios productos por venta y referenciando al detalle general de la venta en específico.

La tabla detalle_venta, fue creada para representar la relación muchos a muchos entre ventas y productos según las reglas de normalización.

```
DROP TABLE IF EXISTS detalle_venta;
CREATE TABLE detalle_venta (
    id_detalle INT AUTO_INCREMENT PRIMARY KEY,
    id_venta INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,
    id_almacen INT NOT NULL,
    FOREIGN KEY (id_venta) REFERENCES venta(id_venta),
    FOREIGN KEY (id_producto) REFERENCES producto(id_producto),
    FOREIGN KEY (id_almacen) REFERENCES almacen(id_almacen)
);
```

Creación tabla transferencia:

Con las comprobaciones de la fase 2, se decidió incluir un mecanismo para gestionar el movimiento de productos entre almacenes, integrando esta tabla, esta tabla ofrece:

- Seguimiento completo del ciclo de vida de una transferencia

- Control de estados (solicitada, aprobada, rechazada, completada, cancelada)

- Registro de empleado que solicita y que aprueba

- Validacion de origen y destino diferentes

```
DROP TABLE IF EXISTS transferencia;
CREATE TABLE transferencia (
    id_transferencia INT AUTO_INCREMENT PRIMARY KEY,
    id_producto INT NOT NULL,
    id_almacen_origen INT NOT NULL,
    id_almacen_destino INT NOT NULL,
    cantidad INT NOT NULL,
    fecha_solicitud DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_completada DATETIME NULL,
    estado ENUM('solicitada', 'aprobada', 'rechazada', 'completada', 'cancelada')
DEFAULT 'solicitada',
    id_empleado_solicita INT NOT NULL,
    id_empleado_aprueba INT NULL,
    FOREIGN KEY (id_producto) REFERENCES producto(id_producto),
    FOREIGN KEY (id_almacen_origen) REFERENCES almacen(id_almacen),
    FOREIGN KEY (id_almacen_destino) REFERENCES almacen(id_almacen),
    FOREIGN KEY (id_empleado_solicita) REFERENCES empleado(id_empleado),
    FOREIGN KEY (id_empleado_aprueba) REFERENCES empleado(id_empleado),
    CHECK (cantidad > 0),
    CHECK (id_almacen_origen != id_almacen_destino)
);
```

Implementación de Triggers en SQL

Durante el desarrollo del sistema, se llegó a la conclusión que era una necesidad controlar diferentes situaciones que pueden ocurrir durante el proceso de venta. Se determinaron tres estados posibles de una transacción, pendiente, completada y cancelada.

Para ser capaces de adaptarnos a este tipo de escenarios se realizó una modificación en la estructura de la tabla venta, agregando el atributo “estado”. Para ayudar a la clasificación de cada transacción según la situación dentro del sistema.

Se utilizó la siguiente instrucción:

```
alter table venta
ADD estado ENUM('pendiente','completada','cancelada')
DEFAULT 'pendiente';
```

Se utilizó el tipo ENUM, para restringir los valores posibles del estado.

- Pendiente: la venta ha sido registrada pero no confirmada.
- Completada: la venta se realizó exitosamente.
- Cancelada: la venta fue anulada.

Con el objetivo de mantener y garantizar la integridad de los datos y realizar automáticamente ciertas operaciones del sistema, se implementaron algunos triggers. Permitiendo validar la información antes de almacenarla y ejecutar automáticamente acciones determinadas cuando ocurren operaciones especificas dentro del sistema.

trigger_actualizar_stock_venta

Propósito: El objetivo de este trigger es el de:

Actualizar el stock cuando una venta es registrada.

```
-- Trigger: Actualizar stock al registrar venta
DROP TRIGGER IF EXISTS trigger_actualizar_stock_venta;
DELIMITER $$
CREATE TRIGGER trigger_actualizar_stock_venta
AFTER INSERT ON detalle_venta
FOR EACH ROW
BEGIN
    UPDATE inventario
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto AND id_almacen = NEW.id_almacen;
END $$
DELIMITER ;
```

Validar stock disponible antes de venta

Propósito: El objetivo de este trigger es el de:

Impide ventas cuando el stock es insuficiente.

```
DROP TRIGGER IF EXISTS trigger_validar_stock;
DELIMITER $$
CREATE TRIGGER trigger_validar_stock
BEFORE INSERT ON detalle_venta
FOR EACH ROW
BEGIN
    DECLARE stock_actual INT;
    SELECT stock INTO stock_actual
    FROM inventario
    WHERE id_producto = NEW.id_producto AND id_almacen = NEW.id_almacen;

    IF stock_actual < NEW.cantidad THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuficiente para esta venta';
    END IF;
END $$
DELIMITER ;
```

trigger_inventario_transferencia

Propósito: Actualiza el inventario automaticamente al completar una transferencia.

```
DROP TRIGGER IF EXISTS trigger_inventario_transferencia;
DELIMITER $$
CREATE TRIGGER trigger_inventario_transferencia
AFTER UPDATE ON transferencia
FOR EACH ROW
BEGIN
    IF NEW.estado = 'completada' AND OLD.estado != 'completada' THEN
        UPDATE inventario
        SET stock = stock - NEW.cantidad
        WHERE id_producto = NEW.id_producto AND id_almacen = NEW.id_almacen_origen;

        UPDATE inventario
        SET stock = stock + NEW.cantidad
        WHERE id_producto = NEW.id_producto AND id_almacen = NEW.id_almacen_destino;
    END IF;
END $$
DELIMITER ;
```

trg_empleado_auth_hash

El propósito de este trigger es: Hashea automaticamente las contrasenas al insertar en la tabla de autenticacion.

```
DROP TRIGGER IF EXISTS trg_empleado_auth_hash;
DELIMITER $$
CREATE TRIGGER trg_empleado_auth_hash
BEFORE INSERT ON empleado_auth
FOR EACH ROW
BEGIN
    SET NEW.password_empleado = SHA2(CONCAT(NEW.password_empleado, 'S1st3m4_L4g4l1'), 256);
END $$
DELIMITER ;
```

Creación de índices

Con el fin de optimizar las consultas del sistema, fueron implementados índices en las columnas utilizadas frecuentemente. Como identificadores de productos, ventas y registros del inventario.

```
CREATE INDEX idx_detalle_venta_idventa ON detalle_venta(id_venta);

CREATE INDEX idx_detalle_venta_idproducto ON detalle_venta(id_producto);

CREATE INDEX idx_inventario_producto ON inventario(id_producto);

CREATE UNIQUE INDEX idx_producto_almacen ON inventario(id_producto,id_almacen);
```

Creación de Vistas y Procedimientos

```
DELIMITER $$

CREATE PROCEDURE sp_registrar_venta(
    IN p_id_empleado INT,
    IN p_id_producto INT,
    IN p_id_almacen INT,
    IN p_cantidad INT
)
BEGIN
    DECLARE v_precio DECIMAL(10,2) DEFAULT NULL;
    DECLARE v_stock INT DEFAULT NULL;
    DECLARE v_total DECIMAL(10,2) DEFAULT NULL;
    DECLARE v_id_venta INT DEFAULT NULL;
    DECLARE v_contador INT DEFAULT 1;
    DECLARE v_existe_empleado INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    IF p_cantidad <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La cantidad debe ser mayor que cero';
    END IF;

    START TRANSACTION;

    SELECT COUNT(*)
    INTO v_existe_empleado
    FROM empleado
    WHERE id_empleado = p_id_empleado;

    IF v_existe_empleado = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El empleado no existe';
    END IF;
```

```
    SELECT precio
    INTO v_precio
    FROM producto
    WHERE id_producto = p_id_producto
    LIMIT 1;

    IF v_precio IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El producto no existe';
    END IF;

    SELECT stock
    INTO v_stock
    FROM inventario
    WHERE id_producto = p_id_producto
      AND id_almacen = p_id_almacen
    LIMIT 1
    FOR UPDATE;

    IF v_stock IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No existe inventario para ese producto en ese almacen';
    END IF;

    IF v_stock < p_cantidad THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuficiente';
    END IF;

    SET v_total = v_precio * p_cantidad;

    INSERT INTO venta (fecha, total, id_empleado)
    VALUES (NOW(), v_total, p_id_empleado);

    SET v_id_venta = LAST_INSERT_ID();

    WHILE v_contador <= p_cantidad DO
        INSERT INTO detalle_venta (id_venta, id_producto)
        VALUES (v_id_venta, p_id_producto);

        SET v_contador = v_contador + 1;
    END WHILE;

    UPDATE inventario
    SET stock = stock - p_cantidad
    WHERE id_producto = p_id_producto
      AND id_almacen = p_id_almacen;

    COMMIT;
END $$

DELIMITER ;
```

Vistas

Reporte de Inventario General

```sql
-- Vista: Reporte de Inventario General
DROP VIEW IF EXISTS vw_reporte_inventario_general;
CREATE VIEW vw_reporte_inventario_general AS
SELECT
    p.id_producto,
    p.codigo_serie,
    p.nombre AS nombre_producto,
    COALESCE(SUM(i.stock), 0) AS stock_actual,
    p.precio AS precio_unitario,
    (COALESCE(SUM(i.stock), 0) * p.precio) AS valor_total
FROM producto p
LEFT JOIN inventario i ON p.id_producto = i.id_producto
GROUP BY p.id_producto, p.codigo_serie, p.nombre, p.precio;

-- Vista: Productos por Ubicacion
DROP VIEW IF EXISTS vw_productos_por_ubicacion;
CREATE VIEW vw_productos_por_ubicacion AS
SELECT
    a.id_almacen,
    a.nombre_almacen,
    a.ubicacion_almacen,
    i.ubicacion_interna,
    p.id_producto,
    p.codigo_serie,
    p.nombre AS nombre_producto,
    i.stock
FROM inventario i
JOIN producto p ON i.id_producto = p.id_producto
```

```
SELECT *
FROM vw_reporte_inventario_general
LIMIT 20;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id_producto	codigo_serie	nombre_producto	stock_actual	precio_unitario	valor_total
1	P0001	Producto 1	314	178.98	56199.72
2	P0002	Producto 2	387	576.59	223140.33
3	P0003	Producto 3	244	371.02	90528.88
4	P0004	Producto 4	311	100.31	31196.41
5	P0005	Producto 5	330	338.49	111701.70
6	P0006	Producto 6	274	391.51	107273.74
7	P0007	Producto 7	224	917.09	205428.16
8	P0008	Producto 8	336	460.92	154869.12

vw_reporte_inventario_general 9 | Read Only

Reporte ventas

```
-- Vista: Reporte de Ventas
DROP VIEW IF EXISTS vw_reporte_ventas;
CREATE VIEW vw_reporte_ventas AS
SELECT
    v.id_venta,
    v.fecha,
    p.id_producto,
    p.codigo_serie,
    p.nombre AS nombre_producto,
    dv.cantidad,
    dv.precio_unitario,
    (dv.cantidad * dv.precio_unitario) AS valor_total,
    e.nombre_empleado,
    a.nombre_almacen
FROM venta v
JOIN detalle_venta dv ON v.id_venta = dv.id_venta
JOIN producto p ON dv.id_producto = p.id_producto
JOIN empleado e ON v.id_empleado = e.id_empleado
JOIN almacen a ON v.id_almacen = a.id_almacen
WHERE v.estado = 'completada';
```

Resumen de transferencias

```sql
-- Vista: Resumen de Transferencias
DROP VIEW IF EXISTS vw_resumen_transferencias;
CREATE VIEW vw_resumen_transferencias AS
SELECT
    t.id_transferencia,
    p.nombre AS nombre_producto,
    ao.nombre_almacen AS almacen_origen,
    ad.nombre_almacen AS almacen_destino,
    t.cantidad,
    t.fecha_solicitud,
    t.estado,
    es.nombre_empleado AS empleado_solicita,
    ea.nombre_empleado AS empleado_aprueba
FROM transferencia t
JOIN producto p ON t.id_producto = p.id_producto
JOIN almacen ao ON t.id_almacen_origen = ao.id_almacen
JOIN almacen ad ON t.id_almacen_destino = ad.id_almacen
JOIN empleado es ON t.id_empleado_solicita = es.id_empleado
LEFT JOIN empleado ea ON t.id_empleado_aprueba = ea.id_empleado;
```

Productos por categorías

```sql
-- Vista: Productos por Categoria
DROP VIEW IF EXISTS vw_productos_por_categoria;
CREATE VIEW vw_productos_por_categoria AS
SELECT
    c.id_categoria,
    c.nombre AS nombre_categoria,
    p.id_producto,
    p.codigo_serie,
    p.nombre AS nombre_producto,
    p.precio
FROM categoria c
JOIN producto p ON c.id_categoria = p.id_categoria
ORDER BY c.nombre, p.nombre;
```

```
SHOW FULL TABLES IN sistema_inventario
WHERE Table_type = 'VIEW';
```

Tables_in_sistema_inventario	Table_type
vw_productos_por_ubicacion	VIEW
vw_reporte_inventario_general	VIEW
vw_reporte_ventas_simuladas	VIEW

Vista de productos por ubicación

```
SELECT *
FROM vw_productos_por_ubicacion
LIMIT 20;
```

id_almacen	nombre_almacen	ubicacion_almacen	ubicacion_interna	id_producto	codigo_serie	nombre_producto	sto
1	Almacen Central	Zona 1	Pasillo 1 - Estante 10	1	P0001	Producto 1	49
1	Almacen Central	Zona 1	Pasillo 1 - Estante 9	2	P0002	Producto 2	56
1	Almacen Central	Zona 1	Pasillo 1 - Estante 2	3	P0003	Producto 3	60
1	Almacen Central	Zona 1	Pasillo 1 - Estante 9	4	P0004	Producto 4	74
1	Almacen Central	Zona 1	Pasillo 1 - Estante 6	5	P0005	Producto 5	40
1	Almacen Central	Zona 1	Pasillo 1 - Estante 1	6	P0006	Producto 6	21
1	Almacen Central	Zona 1	Pasillo 1 - Estante 5	7	P0007	Producto 7	37

Ventas simuladas

```
SELECT *
FROM vw_reporte_ventas_simuladas
LIMIT 20;
```

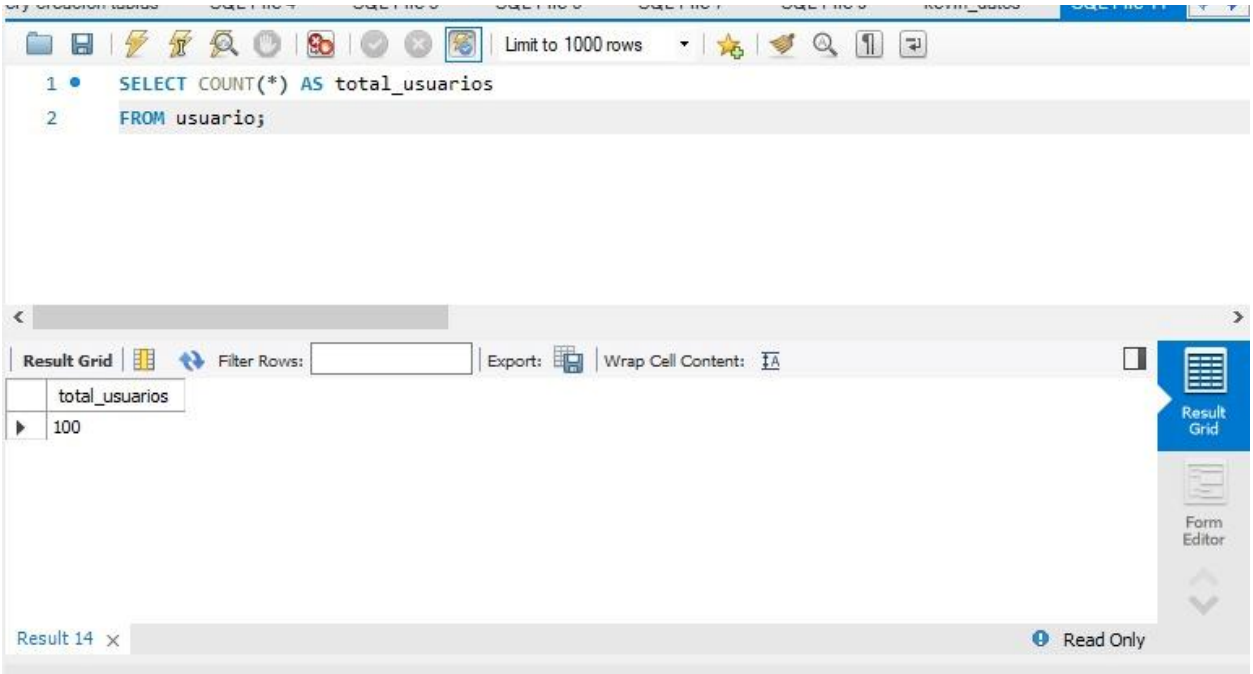
id_venta	fecha	id_usuario	nombre_usuario	id_empleado	nombre_empleado	id_producto	codigo_serie
303	2025-09-12 02:14:27	25	Usuario 25	1	Empleado 1	17	P0017
303	2025-09-12 02:14:27	25	Usuario 25	1	Empleado 1	13	P0013
303	2025-09-12 02:14:27	25	Usuario 25	1	Empleado 1	14	P0014
304	2026-03-06 11:00:27	84	Usuario 84	1	Empleado 1	2	P0002
304	2026-03-06 11:00:27	84	Usuario 84	1	Empleado 1	17	P0017
310	2025-11-21 10:53:27	17	Usuario 17	1	Empleado 1	13	P0013
310	2025-11-21 10:53:27	17	Usuario 17	1	Empleado 1	20	P0020

Muestra de que hay usuarios

```
SELECT *
FROM venta
ORDER BY id_venta DESC
LIMIT 10;
```

id_venta	fecha	total	id_empleado	id_usuario
905	2026-03-09 21:18:38	357.96	1	1
904	2026-03-09 21:18:30	357.96	1	1
903	2026-03-09 21:17:29	357.96	1	1
902	2026-03-09 21:16:34	357.96	1	1
901	2026-03-09 21:16:30	357.96	1	1
900	2026-02-18 23:22:37	2047.53	2	15
899	2025-12-02 08:47:37	503.33	6	43
898	2025-12-12 07:48:37	908.70	1	35

Total de Usuarios



Procedimientos Almacenados

Productos por rango de precio

```sql
-- Productos por rango de precio
DROP PROCEDURE IF EXISTS sp_productos_por_rango_precio;
DELIMITER $$
CREATE PROCEDURE sp_productos_por_rango_precio(IN precio_min DECIMAL(10,2), IN precio_max DECIMAL(10,2))
BEGIN
    SELECT id_producto, codigo_serie, nombre, descripcion, precio
    FROM producto
    WHERE precio BETWEEN precio_min AND precio_max
    ORDER BY precio;
END $$
DELIMITER ;
```

Stock Minimo

```sql
-- SP: Stock minimo
DROP PROCEDURE IF EXISTS sp_stock_minimo;
DELIMITER $$
CREATE PROCEDURE sp_stock_minimo(IN limite INT)
BEGIN
    SELECT
        p.id_producto,
        p.codigo_serie,
        p.nombre,
        i.id_almacen,
        a.nombre_almacen,
        i.stock
    FROM inventario i
    JOIN producto p ON i.id_producto = p.id_producto
    JOIN almacen a ON i.id_almacen = a.id_almacen
    WHERE i.stock <= limite
    ORDER BY i.stock ASC;
END $$
DELIMITER ;
```

Reporte de ventas por periodo

```
-- SP: Reporte de ventas por periodo
DROP PROCEDURE IF EXISTS sp_reporte_ventas_periodo;
DELIMITER $$
CREATE PROCEDURE sp_reporte_ventas_periodo(IN fecha_inicio DATE, IN fecha_fin DATE)
BEGIN
    SELECT
        v.id_venta,
        v.fecha,
        v.total,
        e.nombre_empleado,
        a.nombre_almacen,
        v.estado
    FROM venta v
    JOIN empleado e ON v.id_empleado = e.id_empleado
    JOIN almacen a ON v.id_almacen = a.id_almacen
    WHERE DATE(v.fecha) BETWEEN fecha_inicio AND fecha_fin
    ORDER BY v.fecha DESC;
END $$
DELIMITER ;
```

Productos Mas vendidos

```
-- SP: Productos mas vendidos
DROP PROCEDURE IF EXISTS sp_mas_vendidos;
DELIMITER $$
CREATE PROCEDURE sp_mas_vendidos(IN limite INT)
BEGIN
    SELECT
        p.id_producto,
        p.codigo_serie,
        p.nombre,
        SUM(dv.cantidad) AS cantidad_vendida,
        SUM(dv.cantidad * dv.precio_unitario) AS total_vendido
    FROM detalle_venta dv
    JOIN producto p ON dv.id_producto = p.id_producto
    JOIN venta v ON dv.id_venta = v.id_venta
    WHERE v.estado = 'completada'
    GROUP BY p.id_producto, p.codigo_serie, p.nombre
    ORDER BY cantidad_vendida DESC
    LIMIT limite;
END $$
DELIMITER ;
```

Gestión de Usuarios y Seguridad

Para implementar un mecanismo de control de acceso, se utilizó MySQL para definir dos roles principales:

- Administrador : Usuario con permisos completos sobre la base de datos.
- Operador : Usuario con permisos limitados, enfocado en registrar ventas y consultar información del inventario.

```
create role administrador;
create role operador;
```

Se concedieron los permisos correspondientes a cada rol:

```
Grant all privileges
on sistema_inventario.* to administrador;
```

```
grant select on sistema_inventario.producto to operador;
grant select on sistema_inventario.inventario to operador;
grant select on sistema_inventario.inventario to operador;
grant insert on sistema_inventario.venta to operador;
grant insert on sistema_inventario.detalle_venta to operador;
```

Particiones

Horizontal

Partición por lista (LIST) según id_almacen. En tabla inventarios.
Durante el proceso de análisis, se tomó en cuenta algunas tablas para candidatos del particionamiento, pero se decidió hacer particionamiento por list según el id de la tabla inventario debido a:

Mantiene una diferencia para realizar el mantenimiento individualmente de cada almacén, evitando que todos entren en esa fase al mismo tiempo. Mantiene escalabilidad horizontal ya que cada almacén tiene su propio inventario y finalmente debido a que acelera las consultas por almacén especificado, ya que solo accede a la información de la partición que corresponde.

Vertical

Empleado auth. Se concluyó que una de las mejores candidatas a particionarse, era empleados, ya que contenía contraseña almacenada en la tabla original, se particionó verticalmente, separando los datos que se podrían consultar de un empleado, de los internos para seguridad. El beneficio que se buscó con este particionamiento es tener una mejor seguridad de datos sensibles, dar auditoria aumentada al llevar mayor control, actualizar formas de autenticación sin que se afecte la tabla principal.

- Empleados: Datos operacionales
- Empleados_auth: datos sensibles

MongoDB

Se implementaron tres colecciones:

- HistorialTransacciones
- CambiosProductos
- ComentariosOperadores

Estas colecciones permiten almacenar información histórica no estructurada, útil para aditoria y análisis.

Seleccionamos la base de datos

Nos indica que trabajaremos en la base de datos “sistema inventario”

```
// Seleccionar la base de datos
use sistema_inventario
```

Creamos las colecciones

db.createCollection(“transacciones_historicas”): Crea la colección donde se guardan las transacciones realizadas en el inventario.

db.createColection(“hostorial_modificaciones”): Crea al colección para registrar cambios o ajustes hechos a los productos.

db.createCollection(“comentarios_operadores”): Crea la colección para almacenar observaciones y comentarios de los operadores.

```
// Seleccionar la base de datos
use sistema_inventario

// Crear colecciones
db.createCollection("transacciones_historicas")
db.createCollection("historial_modificaciones")
db.createCollection("comentarios_operadores")
```

Insertamos documentos de prueba

db.transacciones_historicas.insertOne: Inserta una transacción histórica con código de producto, cantidad, fecha y valor total.

db.historial_modificaciones.insertOne: Inserta un registro de modificación indicando que producto se cambió, que ajuste se hizo, cuando y quien lo realizó.

db.comentarios_operadores.insertOne: Inserta un comentario de un operador sobre un producto en específico.

```
// Insertar documentos de prueba en cada colección
db.transacciones_historicas.insertOne({
  codigo_producto: "P001",
  cantidad: 10,
  fecha: ISODate("2025-12-15"),
  valor_total: 500.00
})

db.historial_modificaciones.insertOne({
  codigo_producto: "P002",
  cambio: "Stock ajustado",
  fecha: ISODate("2026-01-10"),
  usuario: "operador1"
})

db.comentarios_operadores.insertOne({
  codigo_producto: "P003",
  comentario: "Producto con baja rotación",
  fecha: ISODate("2026-02-20"),
  operador: "María"
})
```

Realizamos consultas básicas

db.transacciones_historicas.find: Muestra todos los documentos guardados en la colección de transacciones históricas.

db.transacciones_historicas.aggregate: Agrupa las transacciones por producto y calcula el total vendido y el valor acumulado.

db.comentarios_operadores.find: Filtra los comentarios para mostrar únicamente los realizados por la operadora “Maria”

```mongo
// Consultas basicas (querys)
db.transacciones_historicas.find({})   // Ver todas las transacciones

db.transacciones_historicas.aggregate([
  { $group: {
      _id: "$codigo_producto",
      total_vendido: { $sum: "$cantidad" },
      valor_total: { $sum: "$valor_total" }
  }}
])   // Agrupar ventas por producto

db.comentarios_operadores.find({ operador: "Maria" })   // Ver comentarios de un operador especifico
```

Cambios Fase 2

NUEVAS TABLAS AGREGADAS

- CATEGORIA: Se agregó para categorizar los productos. Enfocado en normalización de datos, reportes por segmento y un mejor análisis de ventas por categoría.

- TRANSFERENCIA: Se implementó una tabla para gestionar el movimiento de productos entre almacenes. Incluye control de estados (solicitada, aprobada, rechazada, completada, cancelada), registro del empleado que solicita y del que aprueba, y validaciones a nivel de base de datos para evitar transferencias al mismo almacén.

- EMPLEADO_AUTH: Se separaron los datos de autenticación del empleado en una tabla independiente. Esto implementa particionamiento vertical de datos sensibles, permitiendo almacenar contraseñas hasheadas con SHA2-256, registrar intentos de login y bloquear empleados sin eliminar sus registros.

MODIFICACIONES EN TABLAS EXISTENTES

- PRODUCTO: Se agregó el campo id_categoria como llave foranea, vinculando cada producto a una categoría predefinida. Para eliminar la necesidad de mantener la categoria dentro del producto sin normalización.

- EMPLEADO: Se eliminó el campo password (movido a empleado_auth) y se agregó el campo activo de tipo BOOLEAN. Este nuevo campo permite desactivar empleados sin eliminar sus registros, manteniendo trazabilidad de ventas históricas.

- VENTA: Se agregó el campo id_almacen para identificar en qué almacén se realizó cada venta. Esto permite análisis de ventas por ubicación y mejor trazabilidad.

- DETALLE_VENTA: Se agregaron los campos cantidad (para múltiples unidades por item) y precio_unitario (para registrar el precio al momento de la transacción, manteniendo historial de precios). También se agregó id_almacen para control de stock por ubicación específica.

COMPONENTES ADICIONALES

- VISTAS: Se crearon vw_productos_por_categoria, vw_resumen_transferencias, y se mejoró vw_reporte_ventas para incluir el nombre del almacén.

- PROCEDIMIENTOS ALMACENADOS: Se implementaron sp_productos_por_rango_precio, sp_stock_minimo, sp_reporte_ventas_periodo y sp_mas_vendidos para consultas frecuentes.

- TRIGGERS: Se crearon trigger_actualizar_stock_venta , trigger_validar_stock , trigger_inventario_transferencia, y trg_empleado_auth_hash (hash automático de contraseñas).

- ROLES Y PERMISOS: Se implementó un sistema de gestión de usuarios con sys_admin y sys_operador, aplicando el principio de menor privilegio.

- PARTICIONES: Se implementó partición horizontal de la tabla inventario por id_almacen para mejorar rendimiento de consultas por ubicación.

Inserción de datos en las bases de datos

Se incluyeron datos de pruebas en el dump SQL y el js de MongoDB.

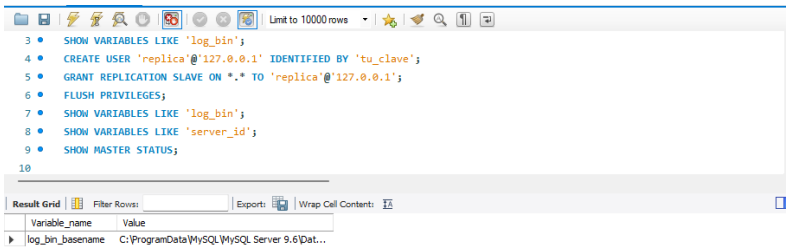
Replicación MySQL

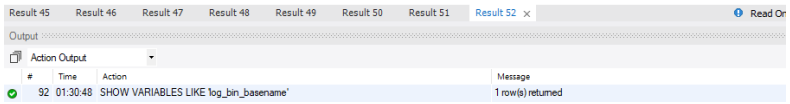
Ajustando my.ini del máster y creando el usuario de replicación.

```
ini

[mysqld]
server_id=1
port=3307
log_bin=mysql-bin
binlog_do_db=sistema_inventario
```

Comandos ejecutados en el Máster

```
SHOW VARIABLES LIKE 'log_bin';

SHOW VARIABLES LIKE 'server_id';

SHOW MASTER STATUS;
```





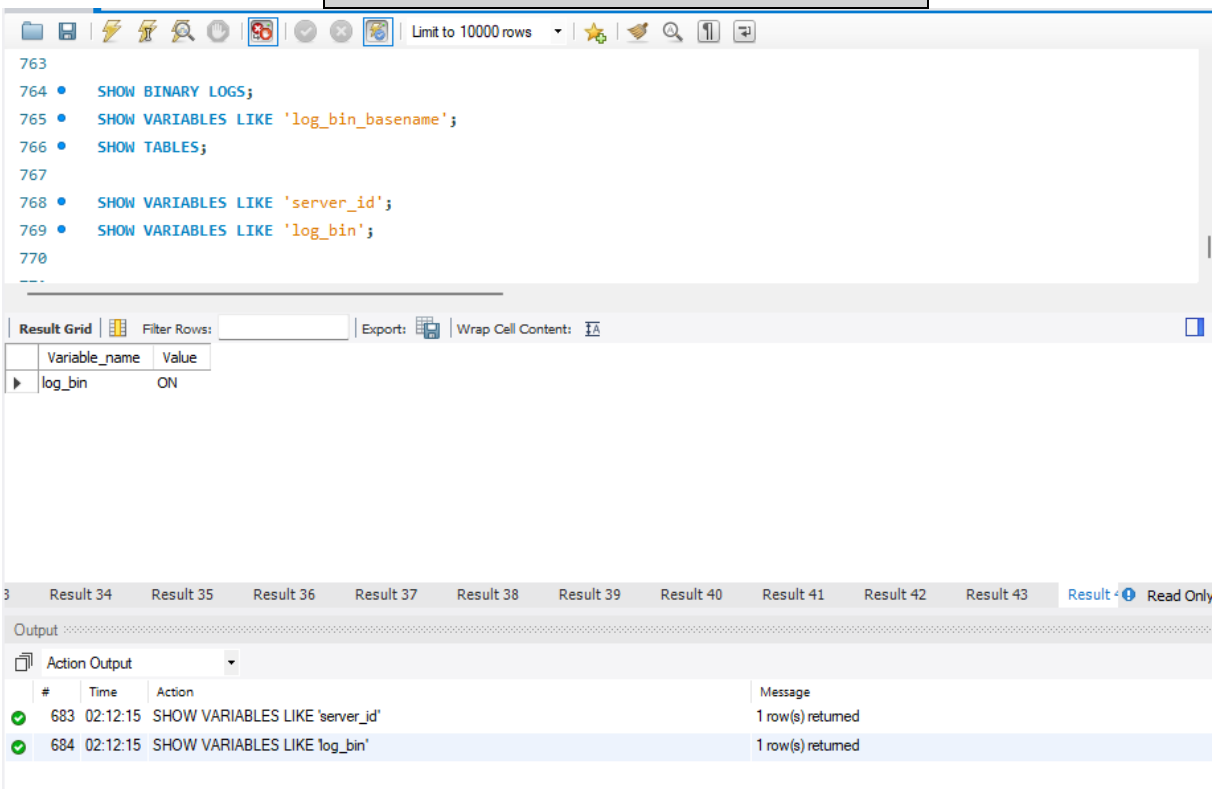
Configuración del Slave

Archivo my.ini del Slave

```
[mysqld]

server_id=2

port=3308

datadir="C:/Program Files/MySQL/MySQL Server 8.0 Slave/data"

log_bin=mysql-bin

relay_log=relay-bin

log_error="C:/Program Files/MySQL/MySQL Server 8.0 Slave/data/mysql-error.log"

bind-address=127.0.0.1
```

Comandos ejecutados en el Slave

```
SHOW VARIABLES LIKE 'server_id';

SHOW VARIABLES LIKE 'log_bin';
```



Creación del usuario de replicación

```
CREATE USER 'replica'@'127.0.0.1' IDENTIFIED BY 'tu_clave';

GRANT REPLICATION SLAVE ON *.* TO 'replica'@'127.0.0.1';

FLUSH PRIVILEGES;
```

```
770 •  CREATE USER 'replica'@'127.0.0.1' IDENTIFIED BY 'tu_clave';
771 •  GRANT REPLICATION SLAVE ON *.* TO 'replica'@'127.0.0.1';
772 •  FLUSH PRIVILEGES;
773
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value
log_bin	ON

Result 52 | Result 53 | Result 54 | Result 55 | Result 56 | Result 57 | Result 58 | Result 59 | Result 60 | Result 61 | Result 6 | Read Onl

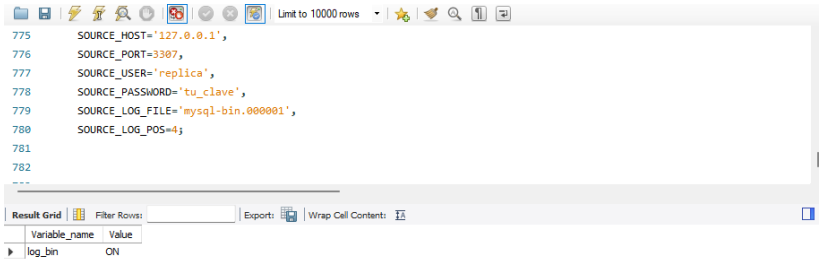
Output

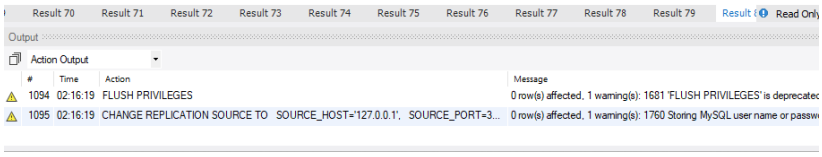
Action Output

#	Time	Action	Message
888	02:14:57	GRANT REPLICATION SLAVE ON *.* TO 'replica'@'127.0.0.1'	0 row(s) affected
889	02:14:57	FLUSH PRIVILEGES	0 row(s) affected, 1 warning(s): 1681 'FLUSH PRIVILEGES' is deprecate

Configuración del Slave para conectarse al Máster

```
CHANGE REPLICATION SOURCE TO

  SOURCE_HOST='127.0.0.1',

  SOURCE_PORT=3307,

  SOURCE_USER='replica',

  SOURCE_PASSWORD='tu_clave',

  SOURCE_LOG_FILE='mysql-bin.000001',

  SOURCE_LOG_POS=4;
```





API

Se incluyó un api con dependencias de:

- Bun – Para crear una build a .exe para fácil ejecución de la API
- Cors = Requerimiento de express para realizar peticiones.
- Dotenv = cargar las variables de entorno para las conexiones a las DB.
- Express = Framework para manejar rutas.
- Mongodb = Driver de Mongo para node.js, permite la conexión y realizar queries a la DB.
- Mysql2 = Driver de MySQL para node.js, permite la conexión y realizar queries a la DB.
- Nodemon = Solo para desarrollo, reinicia cuando detecta cambio en archivos.

Todas incluidas en el package.json.

Se utilizó pool query para realizar las queries, con el objetivo de evitar la saturación de solicitudes a la base de datos, creando varias conexiones simultaneas que permiten limitar las solicitudes, reducir tiempos de espera con las solicitudes y manejar peticiones simultáneamente.

Los endpoints incluidos son:

Productos

GET	`/productos`	Lista Productos
GET	`/productos/:id`	Ver producto en especifico
POST	`/productos`	Crear producto
PUT	`/productos/:id`	Modificar producto
DELETE	`/productos/:id`	Borrar producto

Categorias

GET	`/categorias`	Lista categorias con cantidad de productos
GET	`/categorias/:id`	Ver una categoria con productos
POST	`/categorias`	Crear categoria
PUT	`/caegorias/:id`	Modificar categoria
DELETE	`/categorias/:id`	Borrar categoria

Ventas

GET	`/ventas`	Lista Ventas
GET	`/ventas/:id`	Ver venta en especifico
POST	`/ventas`	Crear venta
PUT	`/ventas/:id`	Cancelar venta

Inventario

GET	`/inventario`	Lista Ventas
GET	`/inventario/:almacenId`	Ver venta en especifico
PUT	`/inventario/:id/:almacenId`	Cancelar venta

Reportes

GET	`/reportes/inventario`	Reporte general (vista sql)
GET	`/reportes/ventas`	Reporte ventas (vista sql)
GET	`/reportes/ubicacion`	Por ubicacion (vista sql)
GET	`/reportes/mas-vendidos`	Ranking productos
GET	`/reportes/bajo-stock?nivel=20`	Stock bajo

MongoDB

GET	`/mongo/transacciones`	Historial de ventas
GET	`/mongo/historial/:codigo`	Historial de cambios de producto
GET	`/mongo/comentarios`	Lista de comentarios
POST	`/mongo/comentarios`	Agregar comentario
DELETE	`/mongo/comentarios/:id`	Eliminar comentario

Justificación técnica

- MySQL: Maneja datos estructurados y relaciones entre entidades.
- MongoDB: Almacena datos históricos flexibles y no estructurados.
- Integración: Ambos motores se complementan para cubrir necesidades operativas y analíticas.

Limitaciones Técnicas

MySQL mantiene varias limitaciones que causaron cambios en el planteamiento de las tablas.

Principalmente la incapacidad de realizar particiones si no es parte de la llave primaria y que la tabla a particionar no tenga llaves foráneas. Para garantizar que la base de datos mantenga la integridad de los datos, se cambió el diseño de la tabla inventario, haciendo una llave primaria compuesta con id_inventario e id_almacen, y se eliminaron las llaves foráneas y se implementó una unique key en id_producto e id_almacen, para que no haya duplicidad de datos, como forma de evitar usar la llave foránea.

Conclusión

El sistema desarrollado permite una gestión eficiente del inventario, combinando tecnologías SQL y NoSQL. La arquitectura garantiza escalabilidad, seguridad y trazabilidad, cumpliendo con los requerimientos del curso simulando un entorno empresarial real.

Presentación

https://drive.google.com/file/d/1s_4nZvLypeCwkE_duKxsBkvLmRlNEsWc/view

https://drive.google.com/file/d/1s_4nZvLypeCwkE_duKxsBkvLmRlNEsWc/view?usp=sharing